

Shiv Nadar Institution of Eminence
Department of Computer Science and Engineering

Behavioral-Diff Detection of Malicious Browser Extension Updates

*A sandboxed, cross-browser runtime-analysis framework for supply-chain threat detection in
browser extensions*

Group Members: Bind Pratap Singh (2310110084), Suhani Deepak Agrawat (2310110717)
Programme: B.Tech CSE, Batch 2023–27
Project Advisor: Dr. Sweta Mishra
Area of Specialization: Cybersecurity
Date: June 15, 2026

Abstract

Browser extensions are a persistent supply-chain risk: a trusted, widely-installed extension can become malicious after a single silent background update, reaching hundreds of thousands of users within hours. Static review fails to catch obfuscated payloads, delayed activation, and behaviours that only surface at runtime. This project proposes a sandboxed *behavioral-diff* framework that executes two consecutive versions of the same extension under identical, automated browsing conditions and computes a weighted delta across their runtime traces. To remove environmental noise, the two versions are driven against *record/replayed* page content so the only meaningful difference is the extension itself. The system actively triggers dormant payloads via synthetic DOM interaction; records network traffic, extension-API calls, DOM access, and storage access; and scores the behavioural change with both a transparent weighted baseline and a Random Forest classifier trained on a synthetic weaponisation corpus and validated on real 2024–25 supply-chain incidents. Crucially, the framework is *cross-browser*: it analyses Chromium-family browsers (Chrome, Edge, Brave) and Firefox, enabling detection of update-time divergence within and across browser engines. The result is an explainable, per-update diff report that is directly actionable for security teams. The system is designed to be implementable within the semester by two final-year CSE students, while leaving a clear novelty surface suitable for a short research paper.

1 Motivation

The browser-extension ecosystem runs on a fundamental *trust contract*: once a user installs an extension, the browser grants it elevated access to every page they visit. This trust contract is **not renewed on each update**. An extension that passes a careful initial review automatically receives the same privileges after every subsequent silent update, with no re-consent from the user and no re-review by the store. The same dynamic exists in *every* major browser.

December 2024 - Cyberhaven Supply-Chain Attack. A threat actor compromised the developer account of Cyberhaven, a legitimate data-loss-prevention

extension with 400,000 active users. A malicious update was pushed to the Chrome Web Store and silently distributed to all users within hours. The payload exfiltrated session cookies and authentication tokens from targeted financial portals - entirely bypassing static review, permission checks, and endpoint security tools. The same campaign is now known to have compromised dozens of additional Chromium extensions.

Static code analysis and permission-manifest inspection cannot catch this class of attack: malicious logic can be obfuscated, condition-gated, or time-delayed. There is currently no open, systematic, *version-aware* tool that detects such behavioural divergence between consecutive extension releases, and existing academic dynamic-analysis tools analyse a *single* version of an extension on a *single* browser. This project targets exactly that gap.

2 Problem Statement

Given two consecutive releases of the same browser extension - version N (trusted baseline) and version $N+1$ (candidate update), packaged as `.crx` (Chromium) or `.xpi` (Firefox) - the system must decide whether the behavioural change between them constitutes a security threat.

The meaningful signal is *change*: new behaviours that did not exist in version N , weighted by their security sensitivity. Formally, the system computes a behavioural delta vector

$$\Delta = f(T_{N+1}) - f(T_N),$$

where T_N and T_{N+1} are runtime traces of each version collected under *identical, replayed* browsing conditions, and $f(\cdot)$ is a feature-extraction function. A scoring function then maps Δ to a risk score and outputs an interpretable diff report explaining which behavioural changes drove the alert. Because the same extension is analysed on multiple browsers, the framework also exposes *cross-engine* divergence (behaviour that appears only under a particular browser's API surface), which is itself a known evasion technique.

3 Proposed Methodology

The system is a five-stage analysis pipeline, parameterised by target browser.

1. **Scope & Sandbox Construction.** Each version of an extension is unpacked and loaded into an isolated Docker container running a headless browser instance - Chromium (covering Chrome, Edge, Brave, which share the Manifest V3 `.crx` format and the `chrome.*` API) and Firefox (`.xpi`, `browser.*` WebExtensions API). All outbound traffic is intercepted by `mitmproxy` (with its CA certificate installed inside the container to terminate TLS). Containers are network-egress-restricted so live malware cannot reach real victims.
2. **Deterministic Browsing & Payload Fuzzing.** To make the diff meaningful, both versions are driven against *the same page content* using a record-and-replay proxy, eliminating ad/nonce/ timestamp noise that would otherwise produce false positives. Over this fixed content, the framework performs synthetic DOM fuzzing - clicking login buttons, populating *dummy* credential forms - across Tranco-ranked top sites, financial login pages, and webmail portals, to activate condition-gated logic. (Tranco replaces the deprecated Alexa list.)
3. **Runtime Trace Collection.** The sandbox records a trace spanning four dimensions: (i) outbound network requests (destinations, payload shape, exfiltration patterns), (ii) DOM access events (reads of form fields, cookies-in-DOM, injected nodes), (iii) storage/cookie access, and (iv) extension-API calls (`chrome.*` / `browser.*`, including MV3 service-worker activity such as `declarativeNetRequest`).

4. **Feature Extraction & Risk Weighting.** Raw traces are normalised and converted into the numerical feature vector Δ . Features are weighted by security sensitivity to manage false positives - e.g., domain-reputation delta, new-destination count, DOM-access intensity, permission delta, and exfiltration-pattern matches.
5. **Scoring: Baseline + Classifier.** Δ is scored two ways for robustness and explainability: (a) a *transparent weighted-rule baseline*, and (b) a *Random Forest* classifier (with XGBoost as a secondary model). Because confirmed malicious *update pairs* are scarce in the wild, the classifier is trained predominantly on a *synthetic weaponisation corpus* - clean extensions into which known malicious behaviours (cookie exfiltration, credential scraping, beaconing) are injected to create labelled $(N, N+1)$ pairs - and is then *validated* on held-out real incidents from the 2024–25 campaigns. The agreement between the interpretable baseline and the learned model is itself reported as a confidence signal.

4 Multi-Browser and Cross-Engine Coverage

A core requirement of this project is coverage across browser families rather than a single browser.

- **Chromium family (primary).** Chrome, Edge, Brave, and Opera share the Manifest V3 `.crx` format and the `chrome.*` API, so a single instrumentation back-end covers all of them with minimal additional work. This is also the family hit by the Cyberhaven campaign.
- **Firefox (second engine).** Firefox uses the `.xpi` package and the `browser.*` WebExtensions API on the Gecko engine. Supporting it requires a separate loader and API-instrumentation layer but yields genuine *cross-engine* coverage and lets the framework detect behaviour that an extension exposes only under one engine.
- **Out of scope.** Safari App Extensions (macOS-only, distinct model) are explicitly excluded.

A shared, browser-agnostic trace schema lets the same feature-extraction and scoring stages operate over both engines, so the cross-engine capability is an architectural property rather than duplicated code.

5 Novelty and Academic Contribution

The project is positioned to balance engineering with a defensible research contribution. The publishable novelty surface is:

- **Update-aware threat formulation.** We analyse the *temporal* supply-chain threat - behavioural divergence between consecutive versions - rather than scoring an extension in isolation, which is what existing dynamic-analysis tools (e.g., Hulk) and IEEE detection work do.
- **Cross-engine behavioral diffing.** To our knowledge, no open framework computes a normalised behavioural diff for the *same* extension across both Chromium and Gecko engines, surfacing engine-specific evasion.
- **Determinism via record/replay.** Diffing two versions over identical, replayed page content isolates the extension as the only variable - a methodological contribution that directly attacks the dominant false-positive source in runtime extension analysis.
- **Explainable, dual-scored output.** A per-update diff report combining an interpretable weighted baseline with a learned classifier produces actionable intelligence (which behaviour changed and why) rather than an opaque label.

If the empirical results hold (high detection on real incidents at low FPR on legitimate updates), this is a natural fit for a short paper / workshop submission in browser or supply-chain security.

6 Implementation and Division of Work

The project relies on an open-source Python stack and divides naturally into two workstreams.

Student A - Bind Pratap Singh (Infrastructure).

- Docker isolation, egress control, and Playwright browser automation (Chromium + Firefox).
- `mitmproxy` traffic interception, the record/replay layer, and DOM/API event instrumentation.
- Browser-agnostic trace-serialisation pipeline and extension-API call logging.

Student B - Suhani Deepak Agrawat (Data & ML).

- Synthetic-weaponisation corpus generation, real-incident dataset curation, labelling, and feature engineering/weighting.
- Weighted-baseline scorer and Random Forest / XGBoost training and cross-validation.
- FastAPI backend service and a Streamlit dashboard rendering the per-update diff report.

7 Evaluation

Metric	Target / Method
Detection Rate (TPR)	Held-out confirmed malicious update pairs from 2024–25 campaigns
False Positive Rate (FPR)	Legitimate updates from top-100 extensions sampled over time
Cross-Browser Consistency	Agreement of the diff verdict across Chromium and Firefox
Evasion Resistance	Obfuscated / delayed-activation variants of known payloads
Per-pair Latency	End-to-end analysis time from extension input to report output
Report Quality	Qualitative: are flagged diff items accurate and actionable?

8 Scope, Phasing, and Risk

Because the full pipeline is substantial for one semester, deliverables are phased so that a working, demonstrable system exists by the end of CSD493, with the remainder forming the follow-on project.

- **Project-1 (this semester) - core, must-have:** sandbox + deterministic browsing + the four-dimension trace pipeline on the *Chromium family*; synthetic corpus; the weighted baseline plus a first Random Forest; an end-to-end diff report on a small set of real incidents.
- **Phase 2 / stretch:** the Firefox (cross-engine) back-end, the Streamlit dashboard, evasion-resistance experiments, and ML refinement toward a paper-grade evaluation.
- **Key risks & mitigations:** (i) *Scarcity of real malicious update pairs* - mitigated by training on the synthetic corpus and reserving real incidents for validation; (ii) *trace non-determinism* - mitigated by record/replay; (iii) *safe malware handling* - mitigated by egress-restricted containers and dummy credentials only.

9 References

References

- [1] D. Bui, B. Tang, and K. G. Shin, “Detection of Inconsistencies in Privacy Practices of Browser Extensions (ExtPrivA),” *IEEE Symposium on Security and Privacy (S&P)*, 2023. [Base paper.] <https://ieeexplore.ieee.org/document/10179338/>
- [2] A. Kapravelos, C. Grier, N. Chachra, C. Kruegel, G. Vigna, and V. Paxson, “Hulk: Eliciting Malicious Behavior in Browser Extensions,” *USENIX Security*, 2014. <https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/kapravelos>
- [3] G. Vasiliadis *et al.*, “WRIT: Web Request Integrity and Attestation Against Malicious Browser Extensions,” *IEEE Transactions on Dependable and Secure Computing (TDSC)*, 2024. <https://ieeexplore.ieee.org/document/10273582/>
- [4] S. Bandhakavi *et al.*, “VEX: Vetting Browser Extensions for Security Vulnerabilities,” *USENIX Security*, 2010. https://www.usenix.org/legacy/event/sec10/tech/full_papers/Bandhakavi.pdf
- [5] A. Aggarwal *et al.*, “Did I Vet You Before? Assessing the Chrome Web Store Vetting Process through Browser Extension Similarity,” arXiv:2406.00374, 2024. <https://arxiv.org/abs/2406.00374>
- [6] S. Karami *et al.*, “Awakening the Web’s Sleeper Agents: Misusing Service Workers for Privacy Leakage,” *NDSS*, 2021. <https://www.ndss-symposium.org/ndss-paper/awakening-the-webs-sleeper-agents-misusing-service-workers-for-privacy-leakage/>
- [7] A. Fass *et al.*, “HideNoSeek: Camouflaging Malicious JavaScript in Benign ASTs,” *ACM CCS*, 2019. <https://dl.acm.org/doi/10.1145/3319535.3363196>
- [8] S. Singh *et al.*, “A Study on Malicious Browser Extensions in 2025,” arXiv:2503.04292, 2025. (Preprint - motivation reference.) <https://arxiv.org/abs/2503.04292>
- [9] “JavaSith: A Client-Side Framework for Analyzing Potentially Malicious Extensions in Browsers, VS Code, and NPM Packages,” arXiv:2505.21263, 2025. <https://arxiv.org/abs/2505.21263>
- [10] V. Le Pochat *et al.*, “Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation,” *NDSS*, 2019. <https://tranco-list.eu/>
- [11] Kaspersky, “Malicious extensions in the Chrome Web Store,” 2023. <https://kaspersky.co.in/blog/dangerous-chrome-extensions-87-million/25861/>